

0.76.13. Un déclencheur en (0, 0) : le D file à droite — en simple

Le 0.76.6 en simple : une ligne de plus, `defile(1, 0, 14, ALPHABET[3], 1, 80)`. Tout le 0.76.6, en quelques lignes.

1. Ce qu'il faut comprendre

C'est le 0.76.6, en simple : une ligne de plus, pour le D : `defile` avec le **numéro 1** et le **sens 1**, vers la droite.

Compare avec le 0.76.6 : là-bas, plus de cinquante lignes, onze variables, trois chronomètres cachés dans les `if`. Ici, quatre lignes dans le bloc `if (actif == 1)`, une par lettre. C'est le même jeu, à l'écran.

Pourquoi avoir appris la version longue : pour savoir ce que ces fonctions font **pour nous**. `tourne_carre` et `defile` contiennent exactement ce qu'on a écrit à la main : un chronomètre, un compteur, des pas.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **3013 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;      // le A part de (10, 0)
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;
```

3. UNE VARIABLE — ACTIF

```
uint8_t actif = 0;    // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

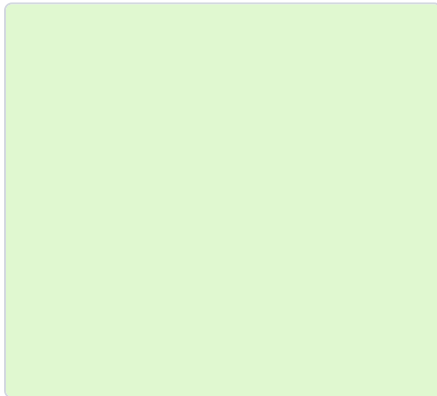
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

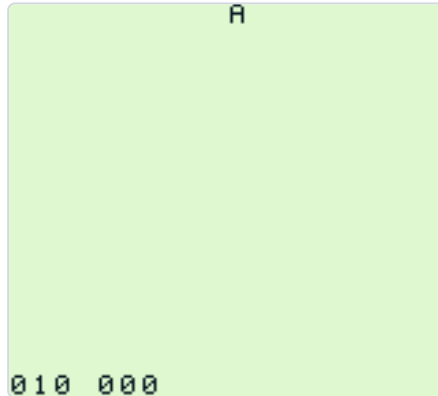
        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 80); // le B tourne en carré
            tourne_carre(1, 14, 12, ALPHABET[1], -4, 80); // le second B, depuis le coin opposé
            defile(0, 19, 3, ALPHABET[2], -1, 80); // le C file à gauche
            defile(1, 0, 14, ALPHABET[3], 1, 80); // le D file à droite
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.76.6 : deux B tournent, un C file à gauche, un D file à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le C va à gauche

✓ le D va à droite, sur la ligne 14

Ensuite : **0.77. Plusieurs rythmes sans compteur : chaque** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++